

# Rapport individuel

Périmètre infrastructure et GitOps

**Auteur**

Tom Patelas

**Client**

Korlea

Mastère DevOps M1, SUP DE VINCI Paris La Défense

Promotion M1DOA 2025-2026

# 1. Ce que je portais dans l'équipe

Dans le projet, je gérais surtout la couche infrastructure : le flake NixOS, le provisionnement des VMs Proxmox, le cluster K3s, la gestion des secrets avec sops-nix, et le bootstrap GitOps via ArgoCD.

L'objectif de mon périmètre était simple : être capable de reconstruire tout le cluster rapidement, de manière reproductible, avec le moins d'intervention manuelle possible.

Concrètement, les principaux éléments que j'ai maintenus pendant le projet étaient :

- le flake NixOS ;
- les modules K3s ;
- les scripts Proxmox ;
- la chaîne sops-nix ;
- le bootstrap ArgoCD ;
- le justfile et les recettes de redéploiement.

C'est aussi le périmètre qui liait tous les autres : si l'infrastructure ne démarrait pas correctement, le travail applicatif, CI/CD ou observabilité ne pouvait pas converger.

## 2. Ce que j'ai réellement fait

### 2.1 Déploiement et reproductibilité

Le sujet principal sur lequel j'ai travaillé pendant tout le projet, c'est la reproductibilité.

Je voulais pouvoir reconstruire un cluster complet à partir de zéro sans avoir à refaire manuellement des étapes de configuration.

Pour ça, j'ai mis en place :

- un flake NixOS pour décrire les nœuds ;
- un pipeline de déploiement via nixos-anywhere ;
- une génération persistante des SSH host keys ;
- une intégration sops-nix pour les secrets ;
- un bootstrap automatique d'ArgoCD.

Aujourd'hui, le cluster peut être recréé avec `just redeploy` en quelques minutes.

Le workflow complet :

1. destruction et création des VMs Proxmox ;
2. installation NixOS via nixos-anywhere ;
3. bootstrap K3s ;
4. déchiffrement automatique des secrets ;
5. installation d'ArgoCD ;
6. synchronisation GitOps des applications.

C'est la partie du projet qui m'a le plus occupé.

## 3. Les principaux problèmes rencontrés

### 3.1 Le bug K3s 1.34

Le problème le plus long à diagnostiquer pendant le projet a été un bug lié à K3s 1.34.

Les nœuds restaient en `NotReady`, sans message d'erreur vraiment exploitable côté Kubernetes.

Le diagnostic final était lié à deux éléments :

- le `cloud-controller-manager` qui crashait ;
- le taint `node.cloudprovider.kubernetes.io/uninitialized` qui restait bloquant.

Le plus difficile dans cet incident n'était pas le fix lui-même.

Le fix tient finalement en quelques flags.

Le vrai sujet, c'était la méthode de diagnostic.

Au début, mon réflexe était surtout de chercher le symptôme exact sur Internet.

Ça n'a pas marché.

Ce qui a réellement débloqué le problème, c'est de lire les logs jusqu'à comprendre moi-même ce qu'ils racontaient.

C'est le plus gros apprentissage technique que je garde du projet.

### 3.2 La trust chain sops-nix

Le deuxième sujet qui m'a occupé, plus court mais plus structurant, c'est la gestion des secrets au premier boot. Sans solution, on retombe sur le classique problème de l'œuf et de la poule : il faut une clé pour déchiffrer, et il faut un nœud installé pour avoir une clé persistante.

Le passage qui a débloqué la chaîne, c'est de dériver les recipients age directement des SSH host keys ed25519 des nœuds via `ssh-to-age`. Les SSH host keys sont générées une fois côté poste développeur, persistées, puis injectées en `extra-files` lors du `nixos-anywhere`. Au premier boot, chaque nœud déchiffre les secrets avec sa propre clé locale, sans intervention humaine.

Une demi-journée à comprendre la séquence exacte d'injection. Le moment où le premier boot a tout déchiffré sans saisie, c'est le truc dont je suis le plus satisfait sur ce projet. C'est ce mécanisme qui rend `just redeploy` réellement automatique.

### 3.3 Les modules virtio dans l'initrd NixOS

Court mais instructif. Au premier `just redeploy` complet, les VMs s'installaient correctement par `nixos-anywhere`, puis rebootaient, et ne remontaient jamais. Pas d'erreur SSH parce qu'il n'y avait rien à atteindre. Pas d'erreur côté `nixos-anywhere` non plus.

La cause : NixOS construit son initrd avec un set minimal de modules par défaut, et n'inclut pas systématiquement les pilotes virtio (`virtio_pci`, `virtio_blk`, `virtio_scsi`, `virtio_net`) requis sur un disque et un réseau virtualisés Proxmox. Correction triviale une fois identifiée : déclarer explicitement les modules dans `nixos/modules/disko-proxmox.nix`.

La leçon est moins triviale. Sur ce type d'environnement, le journal SSH n'a pas le monopole de la vérité. La console Proxmox via `qm terminal` est un outil de diagnostic à utiliser plus tôt que je ne l'aurais cru.

## 4. Ce que je maîtrise aujourd'hui

Aujourd'hui je suis à l'aise sur toute la chaîne infra : NixOS déclaratif, *nixos-anywhere*, le bootstrap et la configuration de K3s, le GitOps avec ArgoCD, sops-nix et le déploiement Proxmox, jusqu'à l'automatisation du redéploiement. Et surtout, je sais déboguer un cluster Linux/Kubernetes qui ne remonte pas.

Au-delà de ces compétences techniques, deux rôles transverses ont structuré mon travail dans l'équipe.

Une posture de lead technique non hiérarchique, conforme à la qualification du DAT §1.2.3 mais sans portée managériale. Concrètement, j'ai conduit les revues de pull requests transverses qui sortaient de mon périmètre (labels des manifests applicatifs, ordonnancement sync-wave de Vault, NetworkPolicy des CronJobs, fix de la startupProbe Grafana), arbitré le découpage NixOS contre ArgoCD, et dépanné l'équipe ponctuellement sur les sujets cluster. Cette posture est venue naturellement parce qu'elle s'appuyait sur le périmètre que je connais le mieux.

Une contribution rédactionnelle au DAT. J'ai rédigé §4.2 (Couche infrastructure NixOS sur Proxmox) de A à Z et la majeure partie de §4.4 (Orchestration K3s y compris le bootstrap *k8s-bootstrap.nix*). Sur l'ensemble du DAT, j'ai rédigé environ un quart des sections moi-même, et relu et corrigé une bonne partie du reste. Relire les sections des autres aussi soigneusement que les siennes en fait partie : un DAT n'a de valeur que si l'équipe entière y reconnaît son travail.

## 5. Ce que je referais différemment

Si je refaisais le projet aujourd'hui, je changerais surtout deux choses.

### 5.1 Automatiser plus tôt

Les procédures répétées auraient dû être automatisées dès qu'elles revenaient régulièrement.

En particulier :

- le bootstrap Vault ;
- le nettoyage Proxmox ;
- certains contrôles de redéploiement.

### 5.2 Documenter plus tôt

Plusieurs procédures sont restées uniquement dans ma tête pendant trop longtemps.

Le problème, c'est qu'une procédure non documentée devient vite difficile à transmettre.

Aujourd'hui, j'essaie de documenter beaucoup plus tôt les tâches répétitives.

## 6. Perspectives d'évolution

Six chantiers que je prioriserais aujourd'hui.

Le stockage arrive en tête. Le provisionner `local-path` par défaut de K3s écrit les PVCs sur le nœud qui planifie le Pod, et un Pod stateful ne migre donc pas entre nœuds. Pour trois VMs, Longhorn est plus adapté que Rook-Ceph : chart Helm simple, réplication à trois répliques par défaut, overhead modéré.

Juste derrière, il manque un backup formalisé de Postgres. Aucun `VolumeSnapshot`, Velero ou backup natif CNPG n'est en place, alors que CNPG sait faire du `backup.barmanObjectStore` vers un stockage S3-compatible, avec WAL archiving continu et base backup quotidien, donc une restauration point-in-time à la minute près.

Vault doit aussi sortir du mode dev. Le root token statique d'aujourd'hui laisserait place à un mode `standalone` avec stockage persistant, un scellement automatique via KMS cloud et un Job d'init idempotent qui crée les paths secrets au boot.

Le control-plane K3s reste un SPOF, avec un seul cp. K3s gère le mode HA embedded etcd à trois cp en quorum : deux VMs de plus, en échange de la tolérance à la perte d'un cp, et dans un contexte client réel la probabilité justifie le coût.

Le chiffrement disque est le sujet où j'ai le plus à apprendre. Disko configure `luks2` en déclaratif, mais la passphrase au boot doit rester compatible avec l'absence d'intervention manuelle, et NBDE (Network Bound Disk Encryption) via Tang me paraît le compromis le plus cohérent.

Reste le durcissement Tailscale. Le service tourne en configuration minimale ; en production, les ACLs Tailscale deviennent indispensables, parce qu'un compte tailnet compromis ne doit pas accéder à toutes les routes par défaut.

Le point le plus important pour moi reste le stockage. Aujourd'hui, `just redeploy` reconstruit le cluster. Mais reconstruire n'est pas sauvegarder. Tant que le stockage n'est pas réellement répliqué, la plateforme reste fragile face à la perte d'un nœud.